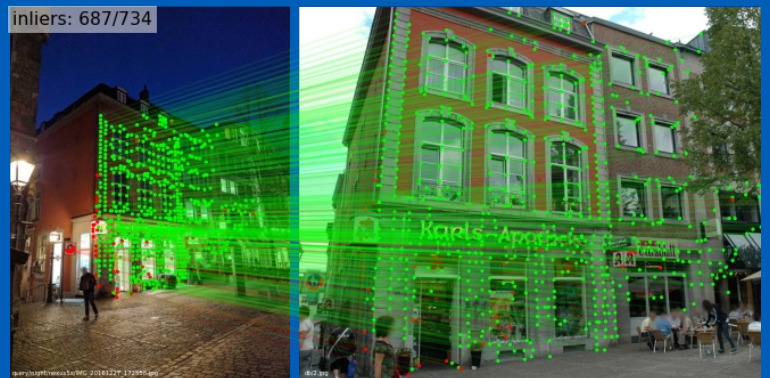




UNIVERSITÀ DI PARMA

Feature Matching

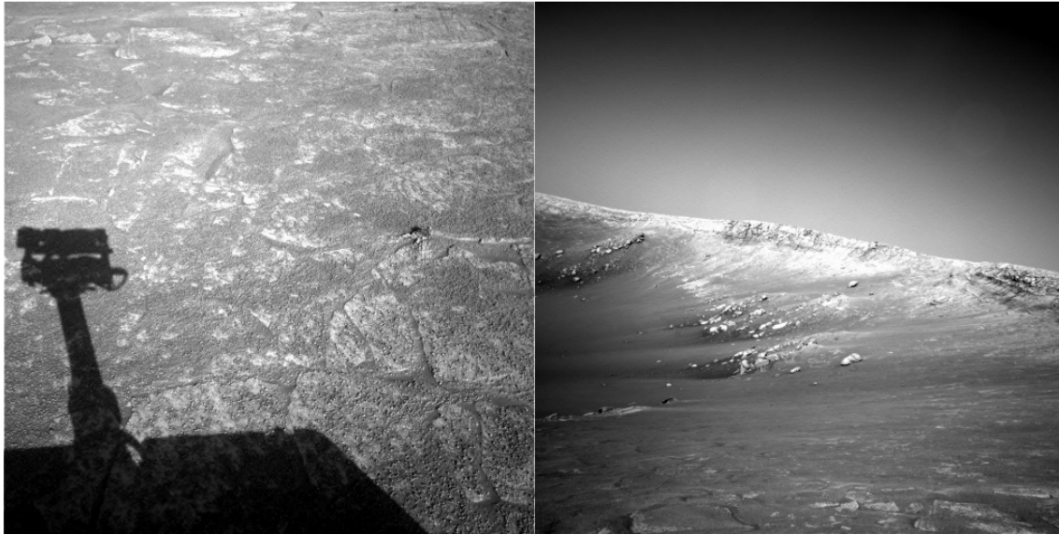


Features Matching

- Keypoints descriptor
 - SIFT
 - BRIEF
 - BRISK
- Performance
- Feature Matching

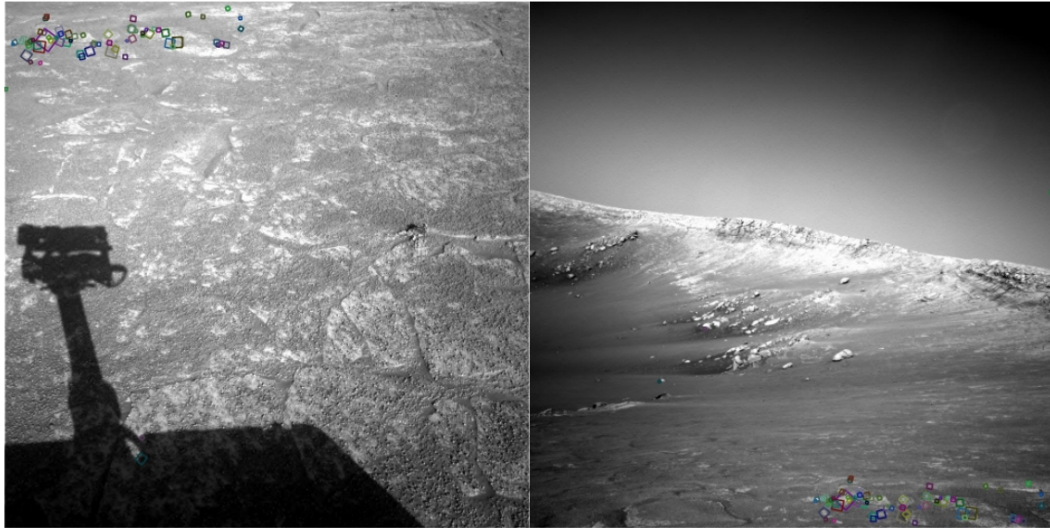
Introdurremo il concetto di descriptor per poi vedere come lo si usa (ci limiteremo al classico image stitching)

- [FP] D. A. Forsyth and J. Ponce. **Computer Vision: A Modern Approach** (2nd Edition). Prentice Hall, 2011.
- **CS231A · Computer Vision: from 3D reconstruction to recognition**, Prof. Silvio Savarese – Stanford University
- **CS131 · Computer Vision: Foundations and Applications**, Prof. Fei-Fei Li – Stanford University
- **TTI Chicago: Computer Vision**, Raquel Urtasun, University of Toronto
- **Elementi di Analisi per Visione Artificiale**
 - Paolo Medici <http://www.ce.unipr.it/people/medici>



NASA Mars Rover images

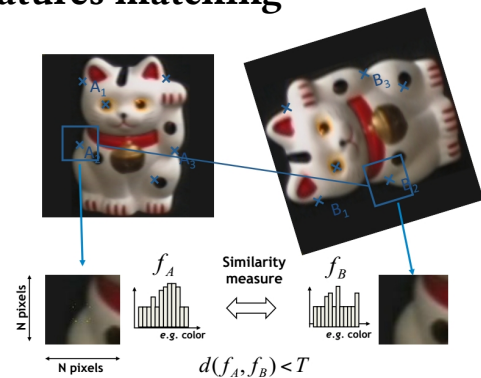
Estraggo feature da diverse immagini. Quelle di una (almeno in parte) le ritrovo nell'altra? E come faccio a capirlo?



NASA Mars Rover images with SIFT feature matches

Sì, almeno in parte, ci si riesce

- Select specific points → keypoints or features → **features extraction**
- Find potential correspondences → **features matching**
- Application dependent step
 - Matching
 - Indexing
 - Detection
 - Image alignment

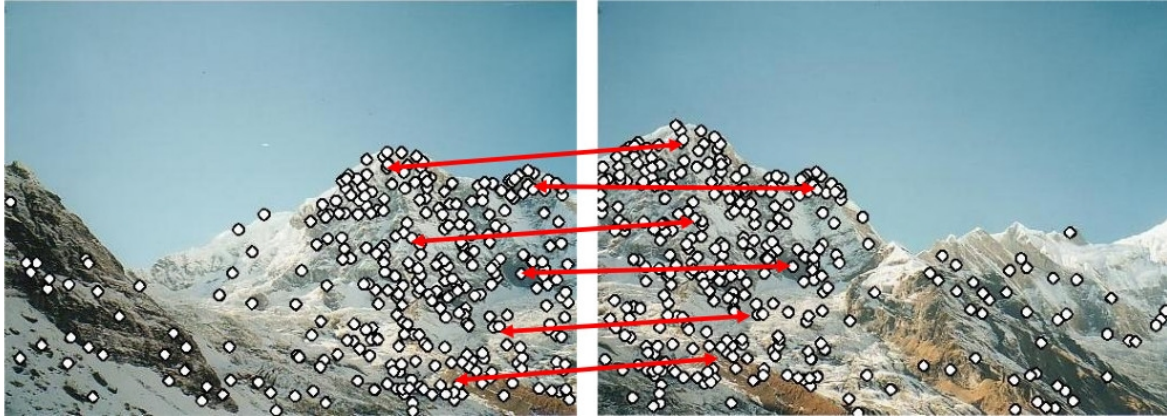


Per come si estraggono ormai è chiaro (a dire il vero abbiamo visto un metodo semplice da spiegarsi ma ne esistono di più efficaci).

Ora concentriamoci solo sulla seconda parte: il matching

La figura del gattino spoilerà un po' tutto

- How to describe keypoints for matching?
 - Keypoints themselves does not work → feature descriptor

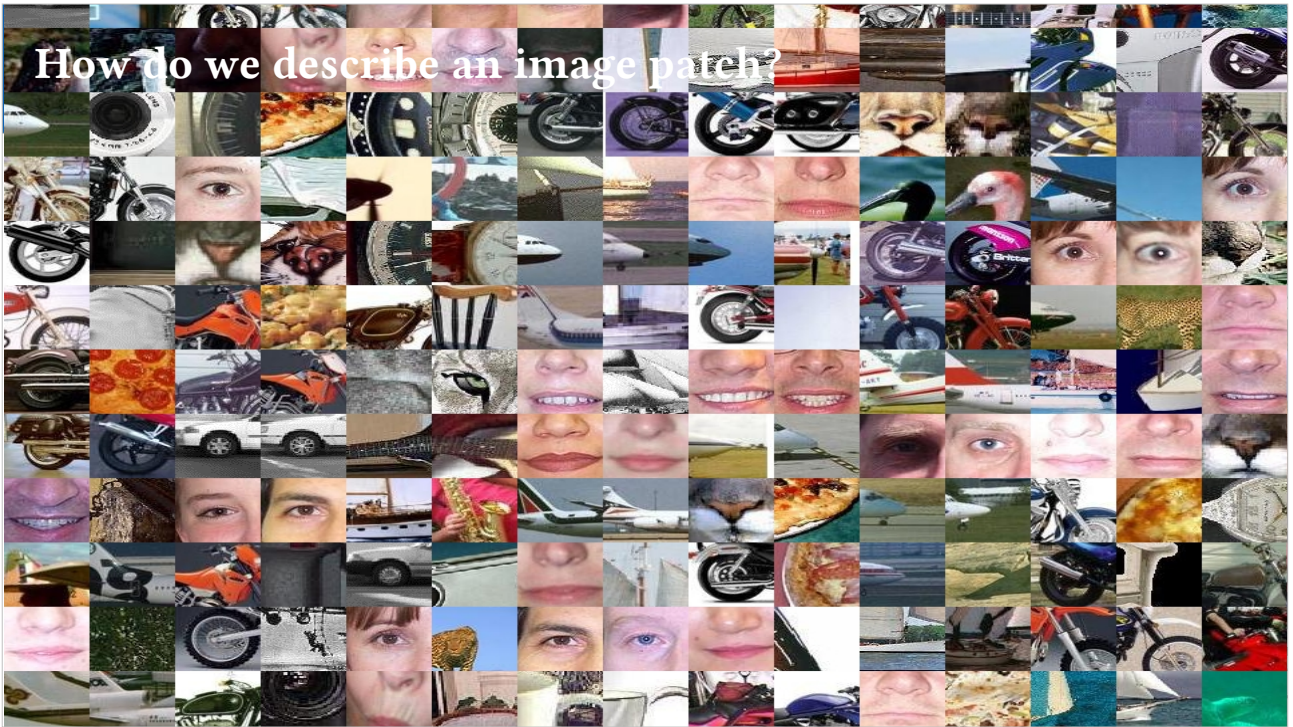


Come faccio il match? Di fatto non posso confrontare i singoli pixel (come già visto nel caso dello stereo). Userò un intorno della feature (le patch).

Però non farò un confronto punto a punto comunque (cosa che viceversa facevo nel caso stereo) ma userò un “descrittore” che mi codifica il contenuto di quella patch.

Serve per distinguere un punto di interesse da un altro in modo da renderlo semi univoco e in modo da poterlo ritrovare in altre immagini

How do we describe an image patch?



- Match should be as much as unique as possible
- Issues
 - Geometrical transformations
 - Translation
 - Scale
 - Rotation
 - ...
 - Photometrical transofrmations
 - Illuminations
 - Color
 - ...

Attenzione che non tutte queste richieste sono ogni volta da soddisfare.
Forte convergenza tra Classificazione e questo settore della computer vision ->
Descrizione delle Regioni



Esempio di match che mi piacerebbe venga fuori (ammesso che riesca ad estrarre da entrambe le immagini quella patch)



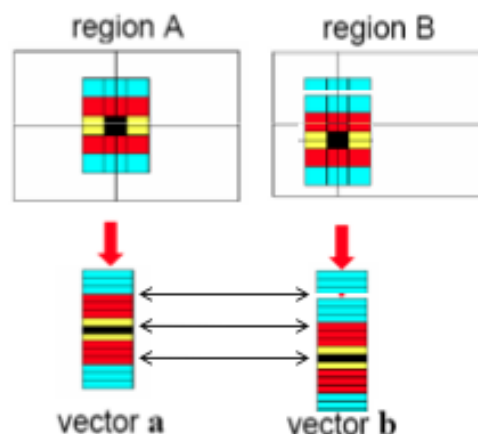
T. Tuytelaars ECCV 2006 tutorial

- To match patches we do not use patches themselves
- What is a keypoint descriptor?
 - Keypoint “measure” that can be effectively used for matching
- Properties
 - Invariant/Robust
 - Distinctive
 - Compact
 - Efficient

Per “misura” si intende numeri, vettori, matrici ecc. Tutte estratte dai pixel di intorno del keypoint secondo diverse strategie



- Create a feature vector, and normalize it
 - Normalization for mean and variance reduce matching problems
- Very sensitive to even small shifts, rotations and any affine transformation

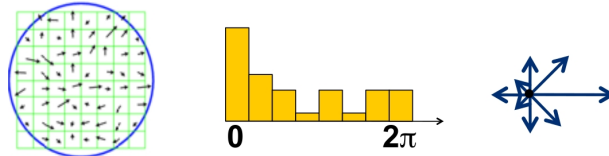


Proviamo a usare le patch “raw” ovvero così come sono. Metto i relativi pixel in fila e si stratta di capire se i due vettori sono facilmente confrontabili...

Spoiler: non lo sono affatto

Tornate all'immagine precedente e vedete subito che non funzionerebbe

- Compute gradients and a histogram of gradients
 - Rotate the patch according to dominant maxima or average



Problema: invarianza alla rotazione? Calcolo orientazione canonica della patch...come? Modulo e fase del gradiente.

È un discorso ortogonale a tutto il resto

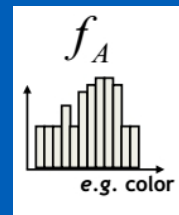
- For Harris keypoints we used a pyramidal approach
 - i.e. a multiscale window





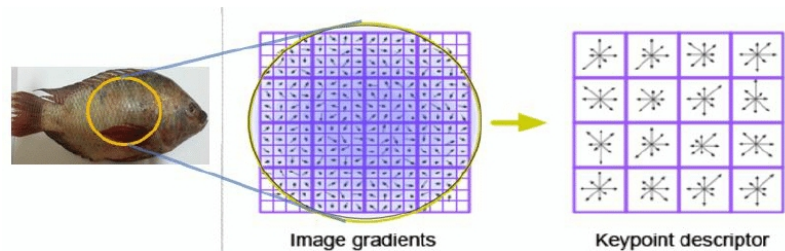
UNIVERSITÀ DI PARMA

Descriptors



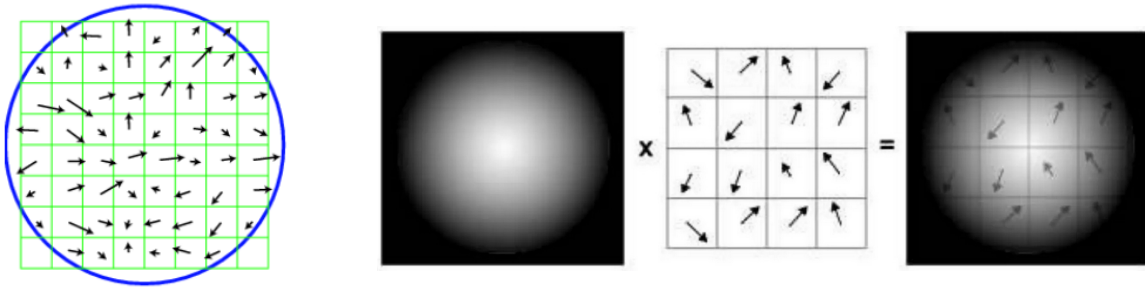
Features Matching

- Lowe 2004
 - Also keypoints extraction
- Robust wrt geometric transformations
 - Scale and Rotation
- Anyway robust wrt luminance variations
- Unfortunately slow



Into: Acronimo Scale Invariant Feature Transform, originariamente pensato come pezzo unico (include anche rilevazione) si può però utilizzare staccato (e per questo viene così descritto), i descrittori di SIFT vanno bene per tutti gli altri Feature Extractor. MA questo vale un po' in generale

- For each keypoint
 - Compute gradient in a specific window around the keypoint
 - Gradient is also downweight using a Gaussian filtering ($\sigma=1.5$ times keypoint size)



16 x 16 dipende dalla scala...

Alla fine quindi ho modulo e fase del gradiente

Il Gaussiano mi porta ad avere una finestra “rotonda”



- For each keypoint
 - Compute gradient in a specific window around the keypoint
 - Gradient is also downweight using a Gaussian filtering ($\sigma=1.5$ times keypoint size)

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y).$$

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

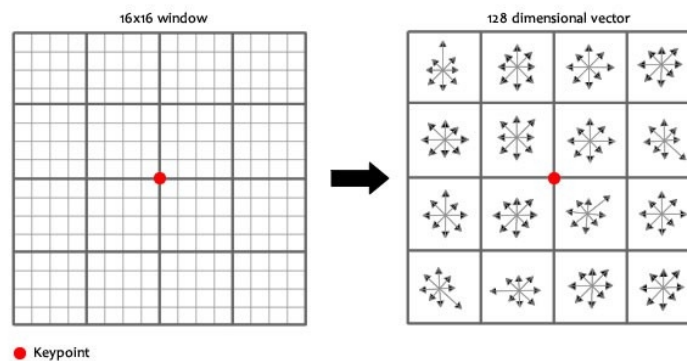
$$\theta(x, y) = \tan^{-1}((L(x, y+1) - L(x, y-1)) / (L(x+1, y) - L(x-1, y)))$$

16 x 16 dipende dalla scala...

Alla fine quindi ho modulo e fase del gradiente

Il Gaussiano mi porta ad avere una finestra “rotonda”

- In the gradient window:
 - Compute a gradient orientation in the 4×4 quadrant



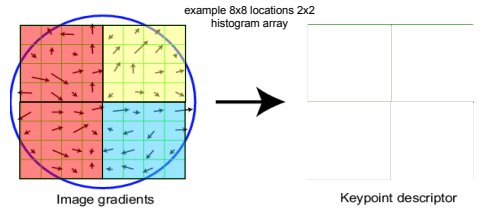
La finestra la dividiamo ulteriormente in quadranti. Per ciascun quadrante abbiamo i gradienti.

Ad esempio se il quadrante è 4×4 ne ho 16. Li quantizzo: 0, 45, 90, ..., 315 e sommo quelli uguali. Il risultato è quello che vedete a dx

Di fatto per ogni quadrante ho un vettore di gradienti.
Qui il vettore è rappresentato in formato grafico.

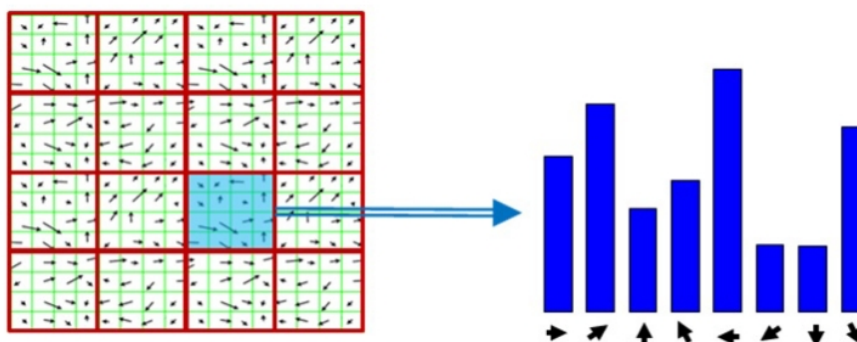
SIFT – Scale Invariant Feature Transform

UNIVERSITÀ
DI PARMA



Per semplicità consideriamo solo 4 quadranti

- In the gradient window:
 - Compute a gradient orientation in each 4×4 quadrant



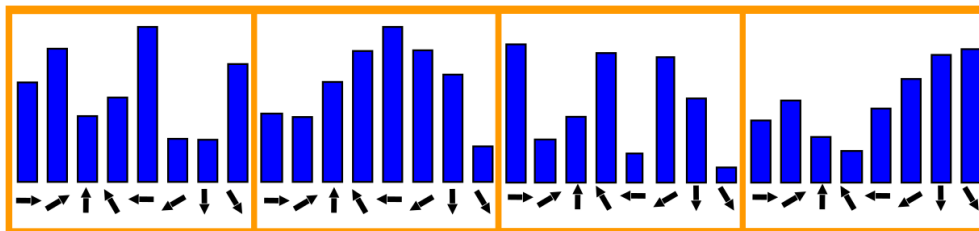
In pratica “serializzo” i risultati. Ottenendo un vettore in cui metto in fila i vettori dei vari quadranti.

Qui ancora una volta è rappresentato graficamente ma di fatto è un vettore che mi codifica:

- indice elemento → quadrante e direzione gradiente
- valore elemento → ampiezza della somma dei gradienti che hanno quella specifica direzione

Vi ricordate come si calcola l’ampiezza? C’è una radice di mezzo e il risultato non è intero → vettore di numeri a virgola mobile

- In the gradient window:
 - Compute a gradient orientation in each 4×4 quadrant
 - Accumulate results in histogram bins (8 in the figure)



In pratica “serializzo” i risultati. Ottenendo un vettore in cui metto in fila i vettori dei vari quadranti.

Qui ancora una volta è rappresentato graficamente ma di fatto è un vettore che mi codifica:

- indice elemento → quadrante e direzione gradiente
- valore elemento → ampiezza della somma dei gradienti che hanno quella specifica direzione

Vi ricordate come si calcola l'ampiezza? C'è una radice di mezzo e il risultato non è intero → vettore di numeri a virgola mobile

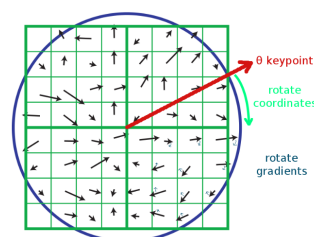
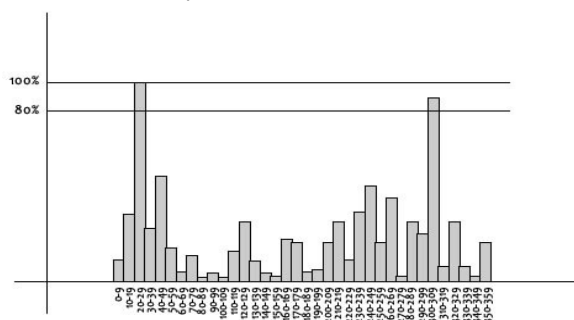


- We showed different situations!
 - Where is the truth?
- Assume that we bin “ o ” orientations over “ $k \times k$ ” quadrants, where each quadrant is “ $s \times s$ ” pixels
- What’s final dimensionality of descriptor?
 - Common implementation: $o=8, k=4, s=4 \rightarrow 128$

Mettiamo qualche numero

Nel caso più comune l’ampiezza del mio vettore è 128 elementi (sembrano pochi ma non lo sono affatto!)

- Maximum gives us info about keypoint orientation
 - Optionally we also consider other potential orientation when we have other values close to maximum
 - Namely we can have more than one descriptor



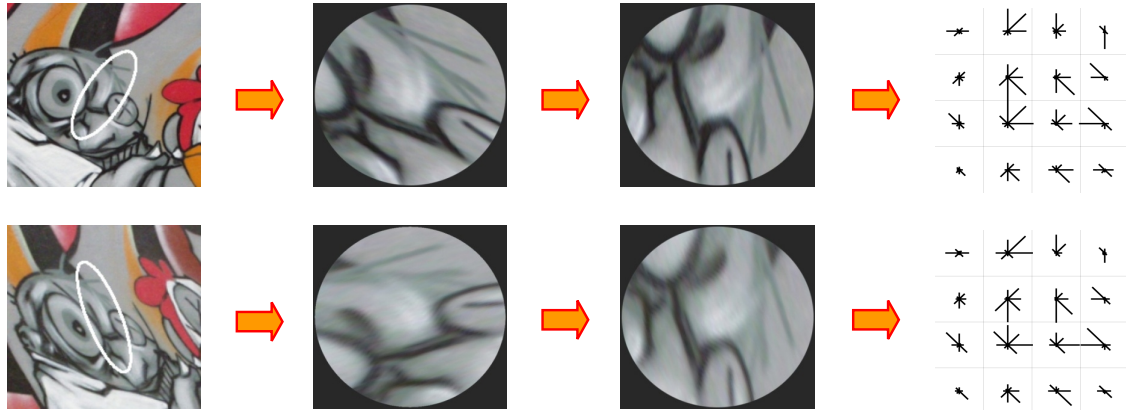
Problema: il descrittore così fatto non è affatto invariante alla rotazione. Come lo affronto?

Ritorno sulla patch senza dividere in quadranti e mi creo un vettore delle varie rotazioni (quantizzato di meno).

Il massimo mi permette di ruotare tutti i vari contributi dei singoli quadranti (di fatto è come ruotare la patch).

Non mi accontento però del massimo, ma prendo anche i picchi che stanno sopra l'80% del massimo

Quindi da una singola patch posso anche ottenere più di un descrittore (è sicuramente una scelta che irrobustisce ma complica un po' il tutto, quindi è opzionale).



Come accennavo è come ruotare la patch.

- How to obtain invariance wrt contrast?
 - Rescale to have unit norm $\rightarrow$
- How to obtain invariance wrt luminance?
 - Do you remember we used gradients?
 - Anyway values are “clipped” against 0.2 to improve robustness to photometric variations
 - Again normalized and the a 0.2 threshold
- How to obtain invariance wrt size?
 - We work on ther relevant pyramid level

$$x = \frac{x}{\|x\|} \quad x \in \mathbb{R}^{128}$$

Clipped to 0.2 significa $x := \min(x, 0.2)$

Szelisky: To further make the descriptor robust to other photometric variations, values are clipped to 0.2 and the resulting vector is once again renormalized to unit length.

However, non-linear illumination changes can also occur due to camera saturation or due to illumination changes that affect 3D surfaces with differing orientations by different amounts. These effects can cause a large change in relative magnitudes for some gradients, but are less likely to affect the gradient orientations.

Therefore, we reduce the influence of large gradient magnitudes by thresholding the values in the unit feature vector to each be no larger than 0.2, and then renormalizing to unit length. This means that matching the magnitudes for large gradients is no longer as important, and that the distribution of orientations has greater emphasis.

- Result is then a 128 dim vector
 - Float values
- How to match them?
 - Euclidean distance can be effectively used
- See OpenCV example

$$dist = \sqrt{\sum_{i=1}^{128} (d[i]_i - d[i]_j)^2}$$

Quindi immaginiamo di voler matchare le feature di una immagine con quelle estratte in altre immagini. A parte che devo considerare di per sé un numero già elevato di match, il match stesso non è leggerino visto che si parla di distanza euclidea



- Summary
 - Extraordinarily robust matching technique
 - Sometimes matches night vs day
 - Lots of code available



Very robust

80% Repeatability at:

10% image noise

45° viewing angle

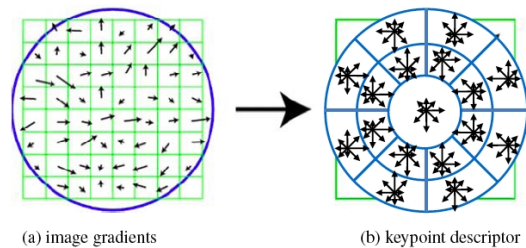
1k-100k keypoints in database

Best descriptor in [Mikolajczyk & Schmid 2005]'s extensive survey

- Principal Component Analysis
 - Reduce the dimensionality of a data set
 - From 128 to 10 or so
 - No quadrants
 - A linear transformation (eigendecomposition) is used to extract principal components
 - Rotation already included
 - Needs training!

Di fatto non divido i quadranti e applico PCA ai gradienti grezzi

- Gradient location-orientation histogram
 - Uses a log-polar binning structure instead of quadrants.
 - Exploits 17 spatial bins and 16 orientation bins.
 - PCA (trained on a large dataset) is used to reduce the 272 space to a 128 one



Anche questa è una variante di SIFT

17 quadranti che non sono proprio quadranti

$17 \times 16 = 272$

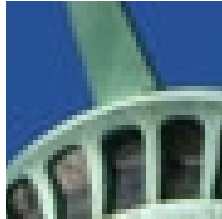
- Speeded Up Robust Feature
 - Again similar to SIFT
 - 20×20 patch subdivided in 4×4 quadrants
 - 64D descriptor
 - As stable as SIFT but several times faster than SIFT

Both SIFT and SURF are patented algorithms!

Prima calcolo rotazione usando patch circolare, poi centro patch quadrata orientata in base al risultato dello step precedente

Notare che sono numeri interi, più facile per calcoli ma non troppo

- Using SIFT, SURF... we obtain “floating point” values
 - Match have to cope with this
 - Binary values can be faster to be matched



[0, 0, 1, 0, 1, 1, 0, 1, ...]

- BRIEF
 - Binary Robust Independent Elementary Features
- FREAK
 - Fast Retina Keypoint
- BRISK
 - Binary Robust Invariant Scalable Keypoints
- ORB
 - Oriented FAST and Rotated BRIEF

Ne vediamo solo alcuni

- After a gaussian smoothing of patch, several intensity comparisons between pair of pixels are performed
- Each pair is selected according to a specific pattern
 - Anyway the same pattern is used for all patches

Binary test

$$\tau(p; x, y) = \begin{cases} 1 & \text{if } p(x) < p(y) \\ 0 & \text{otherwise} \end{cases}$$

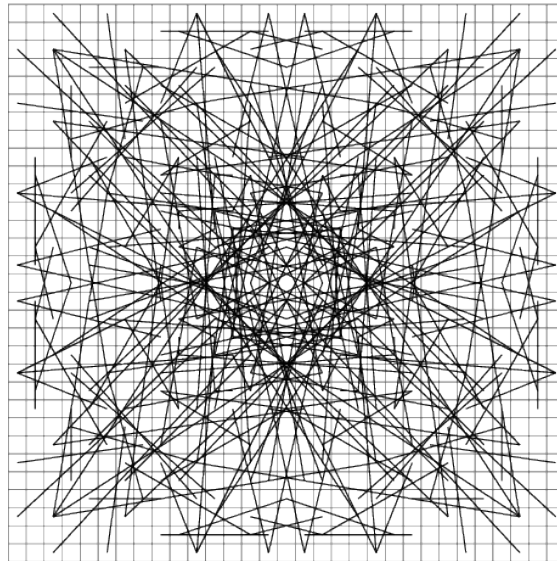
BRIEF descriptor

$$f_{n_d}(p) = \sum_{1 \leq i \leq n_d} 2^{i-1} \tau(p; x_i, y_i)$$

M. Calonder et al. BRIEF: Binary Robust Independent Elementary Features. ECCV 2010.

Tipicamente 128, 256 o 512 coppie

Comunque l'idea di base di BRIEF è quella usata di fatto anche da altri. Invece di calcolare un gradiente come nei casi precedenti mi limito a confrontare coppie di pixel. Assegno 1 o 0 a seconda che un pixel sia più grande o meno dell'altro. Lo possiamo vedere come forma estrema di gradiente...



Ma come decido le coppie che portano al creare il mio descrittore?

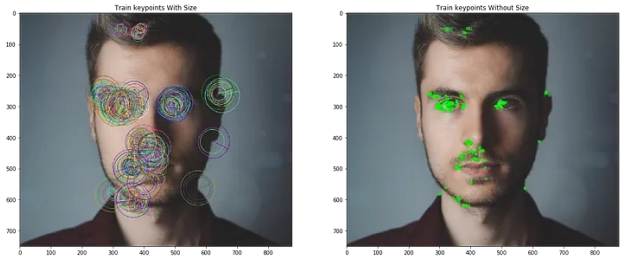
Sono stati proposti tanti possibili pattern, tipicamente 128, 256 o 512 coppie



- Different strategies for sampling geometry
 - Random
 - Uniform: random pairs having anyway a fixed distance
 - Gaussian: gaussian distributions around the keypoint
 - Gaussian 2: first point selected as previous strategy, but second one as gaussian distribution centered on the former
 - Polar: symmetry wrt keypoint
-
- Uniforme: coppie scelte di modo che la loro distanza sia prefissata (di solito emifinestra)
 - Gaussiana: distribuzione gaussiana intorno al keypoint dei due punti
 - Gaussiana 2: primo punto come precedente ma secondo come distribuzione gaussiana intorno al primo
 - Polari: simmetria rispetto al keypoint



- Pro
 - Simple intensity comparison
 - Definitely fast
 - Good results
- Cons
 - Not very reliable against rotations
- Solution → ORB
 - Oriented FAST and rotated BRIEF
 - <https://medium.com/data-breach/introduction-to-orb-oriented-fast-and-rotated-brief-4220e8ec40cf>



ORB nasce in OpenCV

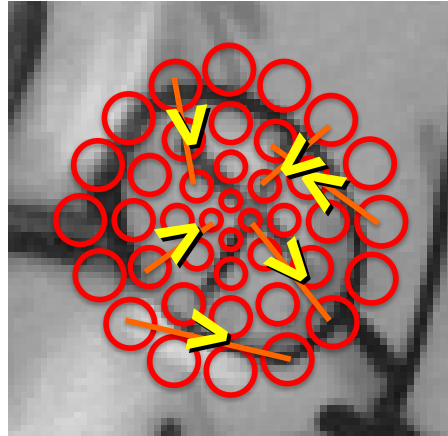
Motivo principale SIFT e SURF sono patented

Comunque **due ordini** di grandezza piu' veloce

<https://medium.com/data-breach/introduction-to-orb-oriented-fast-and-rotated-brief-4220e8ec40cf>

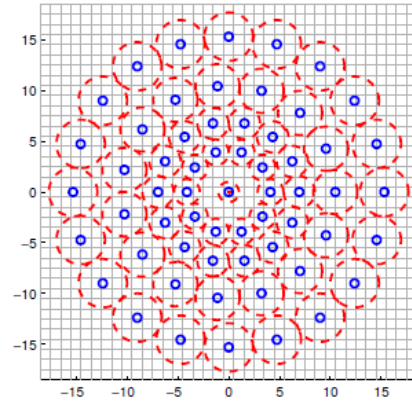
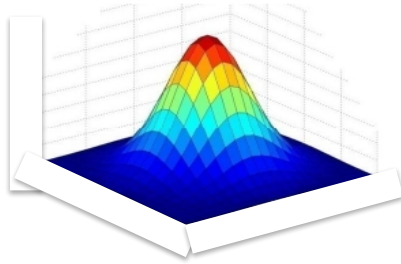


- Binary Robust Scalable Keypoints



- Same approach as BRIEF but given pattern

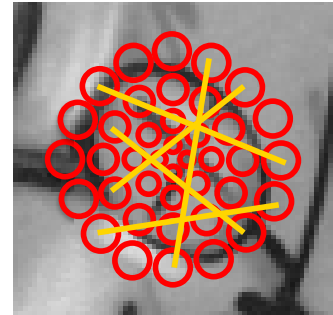
2D Gaussian around each sampling point



Approccio simile a BRIEF, pattern però fisso formato da cerchi circolari e a sua volta è un cerchio

Uso però due tipi di coppie
Distanti per orientazione
Vicine per descrittore

- A number of sampling pairs is selected
- For each pair the gradient is computed between the 2 smoothed intensity
- Different pairs are used for computing descriptor and patch orientation



$$g(p_i, p_j) = (p_j - p_i) \frac{I(p_j, \sigma_j) - I(p_i, \sigma_i)}{\|p_j - p_i\|^2} \quad g = \begin{pmatrix} g_x \\ g_y \end{pmatrix} = \frac{1}{L} \sum_{p_i, p_j \in L} g(p_i, p_j)$$

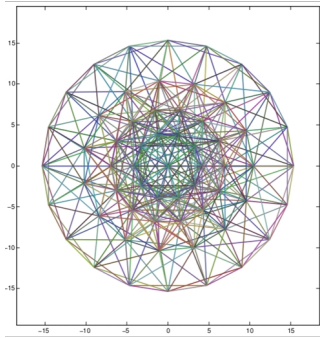
Date le coppie p_i e p_j calcolo gradient locali con la formula mostrata

Where $g(p_i, p_j)$ is the local gradient between the sampling pair (p_i, p_j) , I is the smoothed intensity (by a Gaussian) in the corresponding sampling point by the appropriate standard deviation (see the figure above of BRISK sampling pattern). To compute orientation, we sum up all the local gradients between all the long pairs and take $\arctan(g_y/g_x)$ – the arctangent of the the y component of the gradient divided by the x component of the gradient. This gives up the angle of the keypoint. Now, we only need to rotate the short pairs by that angle to help the descriptor become more invariant to rotation. Note that BRISK only use long pairs for computing orientation based on the assumption that local gradients cancel each other thus not necessary in the global gradient determination.

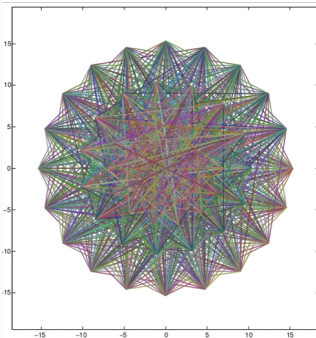


- Short distance pairs $\rightarrow$ descriptor $\rightarrow I(p_j, \sigma_j) > I(p_i, \sigma_i)$
- Long distance pairs $\rightarrow$ orientation $\rightarrow \arctan2(g_y, g_x)$

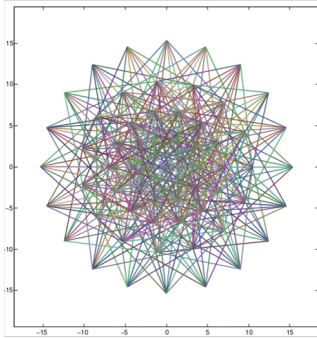
Short-distance pairs (512)



Long-distance pairs (870)



Unused pairs (388)



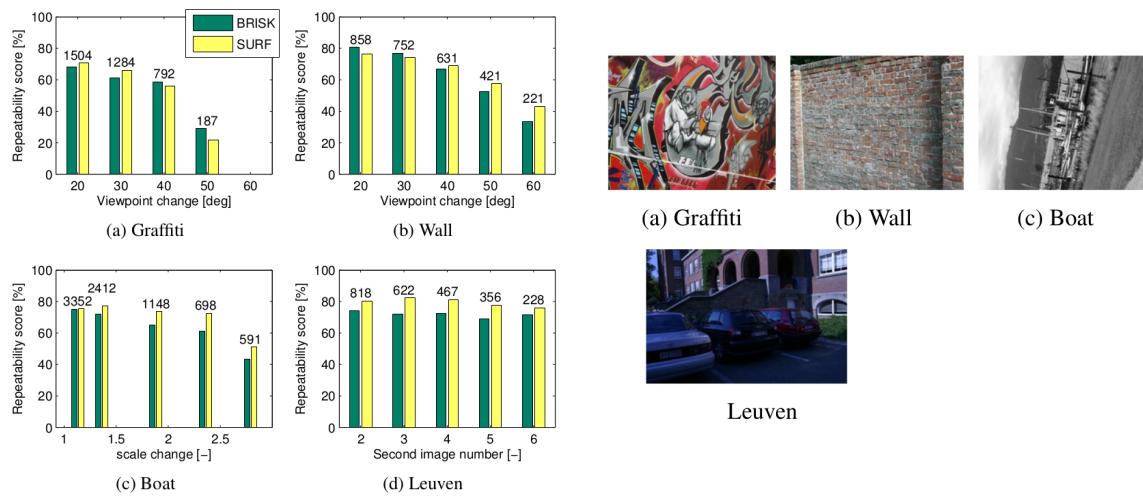


Figure 5. Repeatability scores for 50% overlap error of the BRISK and the SURF detector. The resulting similarity correspondences (approximately matched between the detectors) are given as numbers above the bars.

Leuven è preso con differenti luminosità



- Scalable for faster execution by
 - Reducing number of sampling pairs
 - Examining only one scale
 - Omitting pattern rotation step

	SIFT	SURF	BRISK
Detection threshold	4.4	45700	67
Number of points	1851	1557	1051
Detection time [ms]	1611	107.9	17.20
Description time [ms]	9784	559.1	22.08
Total time [ms]	11395	667.0	39.28
Time per point (ms)	6.156	0.4284	0.03737

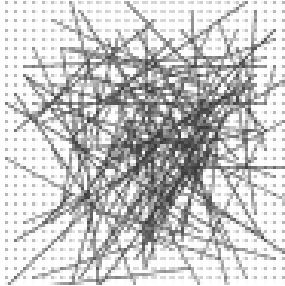
Table 1. Detection and extraction timings for the first image in the Graffiti sequence (size: 800×640 pixels).

	SIFT	SURF	BRISK
Points in first image	1851	1557	1051
Points in second image	2347	1888	1385
Total time [ms]	291.6	194.6	29.92
Time per comparison [ns]	67.12	66.20	20.55

Table 2. Matching timings for the Graffiti image 1 and 3 setup.

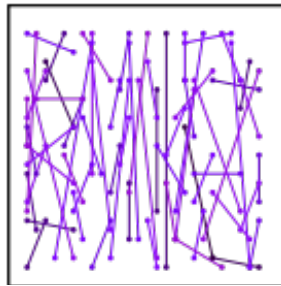


BRIEF



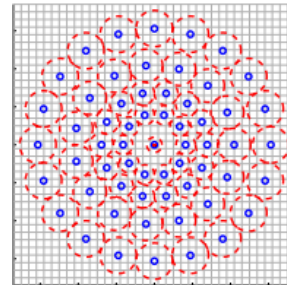
- Efficient

ORB



- Efficient
- Rotation

BRISK



- Efficient
- Rotation
- Scale
- Noise



Autonomous Systems Lab

ETH

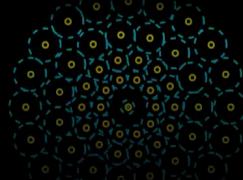
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

UNIVERSITÀ
DI PARMA



BRISK

Binary Robust Invariant Scalable Keypoints



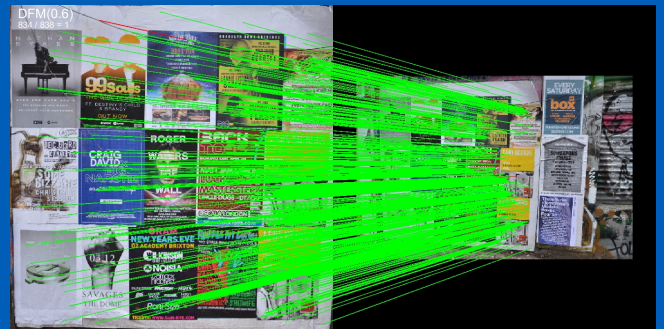
Stefan Leutenegger, Margarita Chli and Roland Siegwart

ICCV 2011



UNIVERSITÀ DI PARMA

Feature Matching



Matching

- We found keypoints
- We compute descriptors
- And now?



Ho la feature i-esima di qua e la j-esima di là. Voglio capire se rappresentano lo stesso punto.

Confronto i descrittori come già detto, ma vediamo nel dettaglio

- For each descriptor i of image A
 - Measure “distance” with each descriptor j of image B
 - Get the two best matches
 - Compare them against a threshold
 - Above $\rightarrow$ no match
 - Below $\rightarrow$ compute the ratio distance

$$\frac{\text{dist}(f_A^i, f_B^{\text{best}})}{\text{dist}(f_A^i, f_B^{2^{\text{nd}} \text{ best}})}$$

In questo caso tengo sotto la soglia in quanto la distanza deve essere la minima

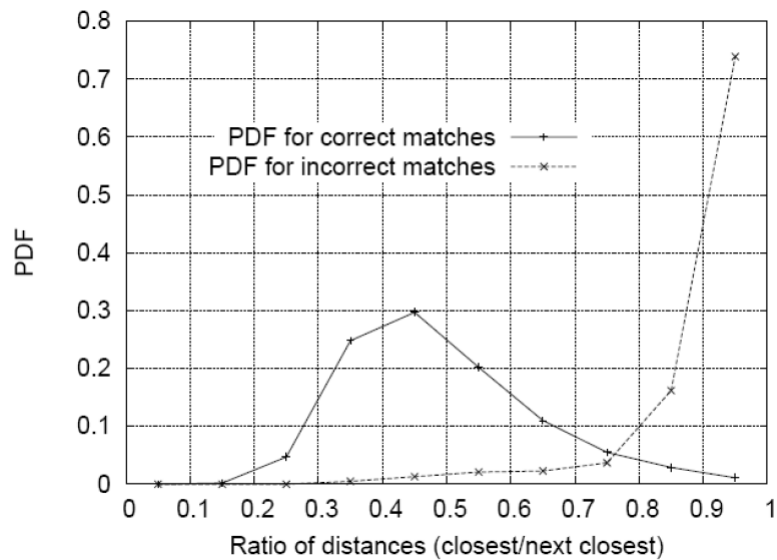
Se ho piú valori sotto soglia mi calcolo il rapporto tra il minimo che ho trovato e il secondo minimo

- When ratio distance $\sim 1 \rightarrow$ no match
 - Matches are too similar $\rightarrow$ ambiguity
- When ratio distance $\ll 1 \rightarrow$ we have a match
 - First match outperforms 2nd one (and others)

$$\frac{\text{dist}(f_A^i, f_B^{\text{best}})}{\text{dist}(f_A^i, f_B^{2^{\text{nd}} \text{ best}})}$$

Voglio solo un minimo che sia significativamente piú piccolo degli altri minimi al fine di scartare ambiguità.

Ok ma quel $\ll$ come lo contestualizziamo?



Using a database of 40,000 keypoints, the solid line shows the PDF of this ratio for correct matches, while the dotted line is for matches that were incorrect

Diciamo che una soglia a 0,7 in questo caso mi salva (quindi un << poi neanche tanto clamoroso). Ovvio che qualche match falso rimane ma sono in ampia minoranza.

Qui è stata usata distanza euclidea



UNIVERSITÀ DI PARMA

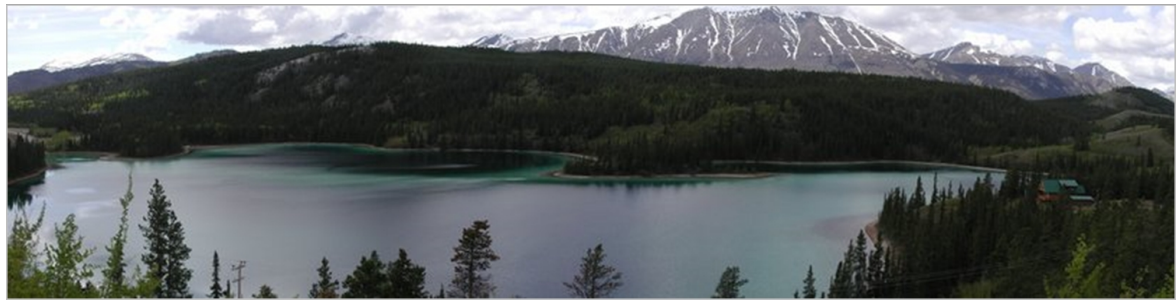
Case of study: Image Stitching



Matching



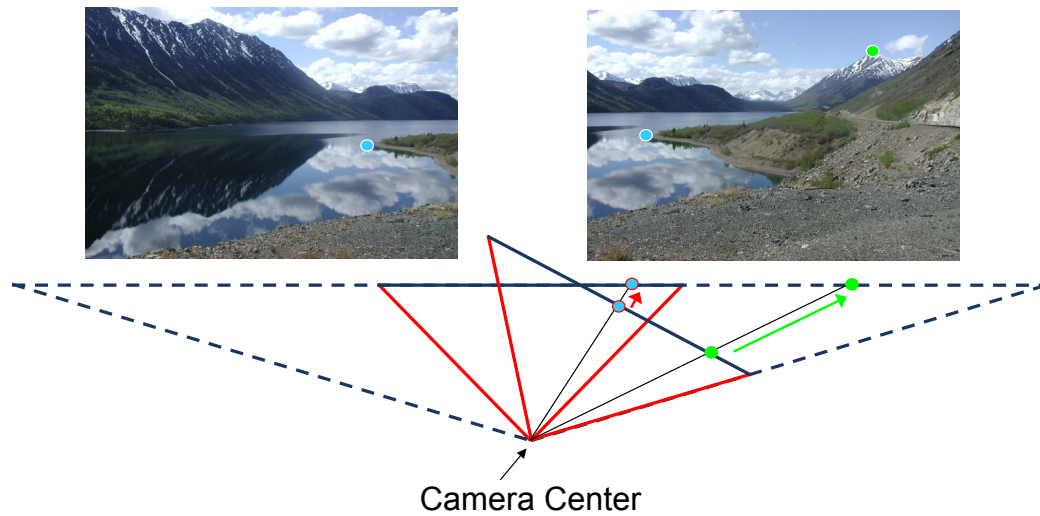
- Combine multiple images in a single view





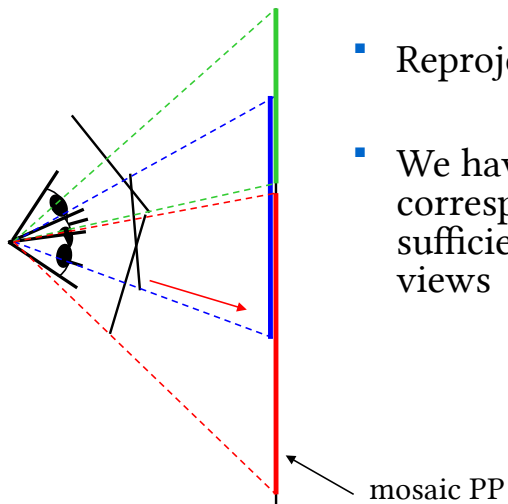
- Main steps
 - Acquire multiple images
 - Get transformation from first image to following one
 - Reproject that image onto the previous one
 - Blend them in the result
 - Until we have images $\rightarrow$ repeat

- Main steps
 - Acquire multiple images
 - Get transformation from first image to following one
 - Reproject that image onto the previous one
 - Blend them in the result
 - Until we have images → repeat



Il fatto che il punto di osservazione sia lo stesso non è poi così rilevante. Funziona benissimo anche nel caso generale.

Però si vede bene come la presenza di punti comuni nelle due immagini mi leghino le immagini stesse. Geometricamente è evidente che riesco a relazionarle.



- Reproject images on a common plane
- We have a “virtual” camera that corresponds to that plane with a FOV sufficiently large to include all other views

L'obiettivo è riproiettare tutte le immagini acquisite (le linee nere) su di un piano comune

- The relation under a projection is an homography

$$\overline{x_2} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}}_H \cdot \overline{x_1}$$

Per riproiettare in un piano comune c'è la solita omografia

- How many matches we need?
 - 9 unknowns? $\rightarrow$ no, we will never be able to solve scale
 - 8 unknowns $\rightarrow$ yes!
- Each match $\rightarrow$ 2 equations
- We need (at least) 4 correspondencies!

$$h = \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix}$$



Mi bastano, teroricamente, 4 corrispondenze

- We can solve using:

$$\begin{cases} x_2 = h_{11}x_1 + h_{12}y_1 + h_{13} \\ y_2 = h_{21}x_1 + h_{22}y_1 + h_{23} \\ z_2 = h_{31}x_1 + h_{32}y_1 + h_{33} \end{cases}$$

- For Euclidean coordinates

$$\begin{cases} x_2' = \frac{x_2}{z_2} = \frac{h_{11}x_1 + h_{12}y_1 + h_{13}}{h_{31}x_1 + h_{32}y_1 + h_{33}} \\ y_2' = \frac{y_2}{z_2} = \frac{h_{21}x_1 + h_{22}y_1 + h_{23}}{h_{31}x_1 + h_{32}y_1 + h_{33}} \end{cases}$$



- 2nd step

$$\begin{cases} x_2'(h_{31}x_1 + h_{32}y_1 + h_{33}) = h_{11}x_1 + h_{12}y_1 + h_{13} \\ y_2'(h_{31}x_1 + h_{32}y_1 + h_{33}) = h_{21}x_1 + h_{22}y_1 + h_{23} \end{cases}$$

- Ordering

$$\begin{cases} -h_{11}x_1 - h_{12}y_1 - h_{13} + h_{31}x_1x_2' + h_{32}y_1x_2' + h_{33}x_2' = 0 \\ -h_{21}x_1 - h_{22}y_1 - h_{23} + h_{31}x_1y_2' + h_{32}y_1y_2' + h_{33}y_2' = 0 \end{cases}$$

- This for a single match

$$\begin{cases} -h_{11}x_1 - h_{12}y_1 - h_{13} + h_{31}x_1x_2' + h_{32}y_1x_2' + h_{33}x_2' = 0 \\ -h_{21}x_1 - h_{22}y_1 - h_{23} + h_{31}x_1y_2' + h_{32}y_1y_2' + h_{33}y_2' = 0 \end{cases}$$

- For all

$$A * h = 0$$

A è quella che mi genera tutte le equazioni dei match avendo quindi
 $Ah=0$



- This for a single match

$$\begin{cases} -h_{11}x_1 - h_{12}y_1 - h_{13} + h_{31}x_1x_2' + h_{32}y_1x_2' + h_{33}x_2' = 0 \\ -h_{21}x_1 - h_{22}y_1 - h_{23} + h_{31}x_1y_2' + h_{32}y_1y_2' + h_{33}y_2' = 0 \end{cases}$$

- For all

$$A = \begin{bmatrix} -x_1^{(1)} & -y_1^{(1)} & -1 & 0 & 0 & 0 & x_1x_2'^{(1)} & y_1x_2'^{(1)} & x_2'^{(1)} \\ 0 & 0 & 0 & -x_1^{(1)} & -y_1^{(1)} & -1 & x_1y_2'^{(1)} & y_1y_2'^{(1)} & y_2'^{(1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ -x_1^{(n)} & -y_1^{(n)} & -1 & 0 & 0 & 0 & x_1x_2'^{(n)} & y_1x_2'^{(n)} & x_2'^{(n)} \\ 0 & 0 & 0 & -x_1^{(n)} & -y_1^{(n)} & -1 & x_1y_2'^{(n)} & y_1y_2'^{(n)} & y_2'^{(n)} \end{bmatrix}$$

A è quella che mi genera tutte le equazioni dei match avendo quindi
HA=0



- Again we can use SVD decomposition to find best fit

$$A = UDV^T$$

- Last column of V is the solution

- Both OpenCV and Eigen provide a SVD solver:

```
cv::SVD::compute(const Mat & A, Mat & D, Mat & U, Mat & Vt)
```

- Not enough we need to normalize result

- Main steps
 - Acquire multiple images
 - Get transformation from first image to following one
 - Reproject that image onto the previous one
 - Blend them in the result
 - Until we have images → repeat



- We now have H
- Simply use

$$\overline{x_2} = H \cdot \overline{x_1}$$

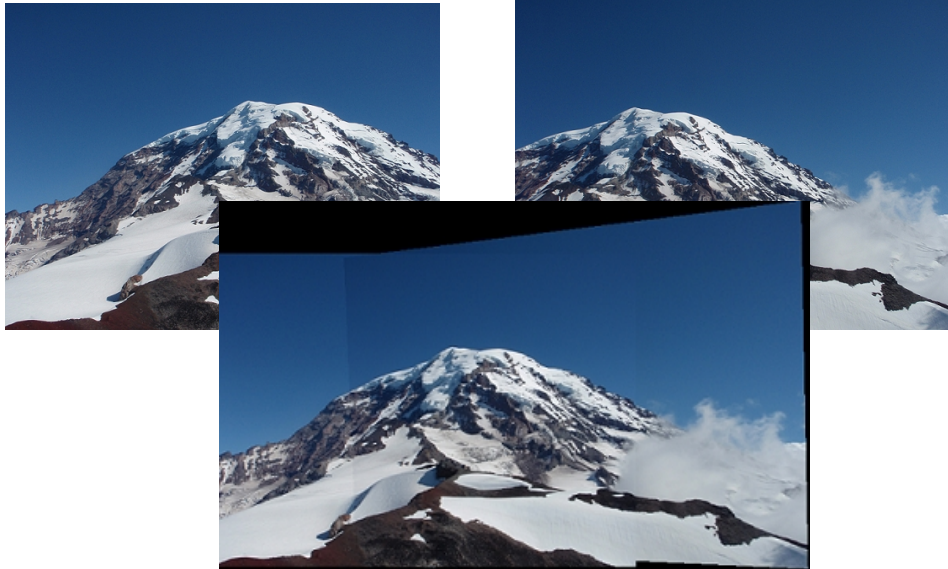
- Basically we fill plane of first image adding other pixels

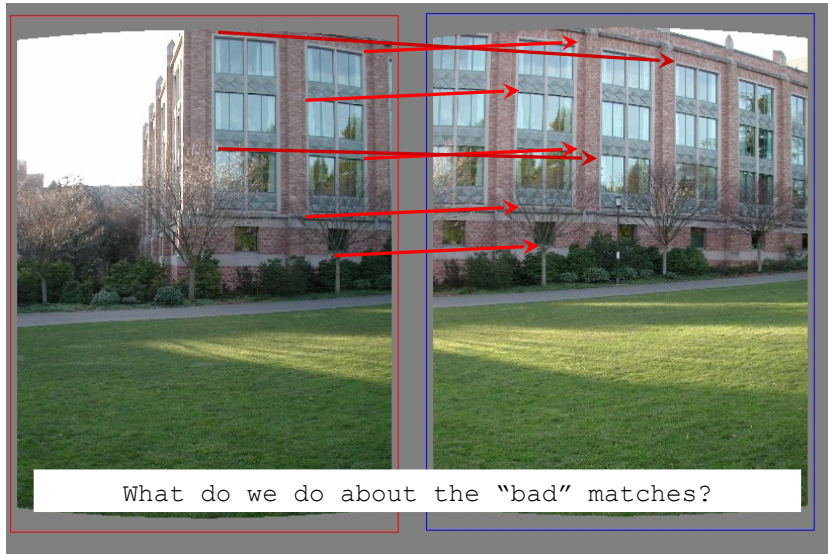
Sì, l'ordine è giusto. Dato un pezzo che voglio riempire nell'immagine 1 lo cerco nella 2

- Main steps
 - Acquire multiple images
 - Get transformation from first image to following one
 - Reproject that image onto the previous one
 - Blend them in the result
 - Until we have images → repeat

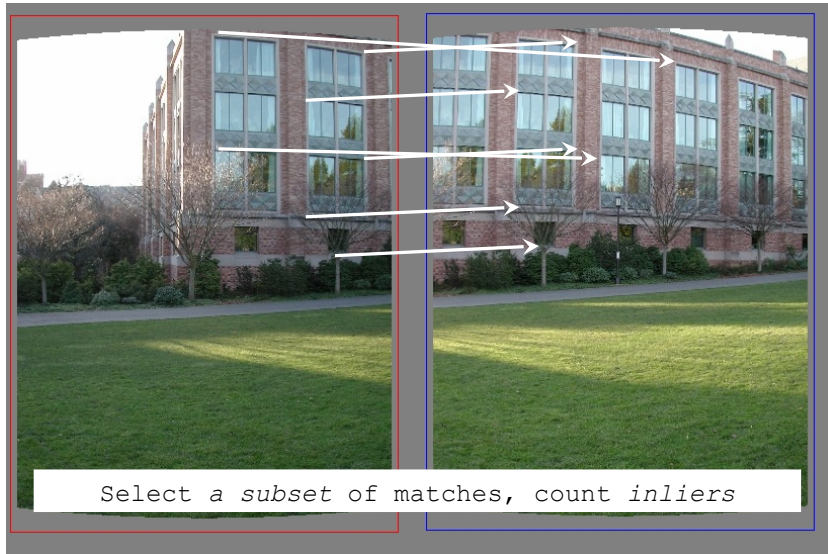


- Consider bilinear interpolation
 - Previous equation gives floating point results
- When we have multiple points from different images use an average



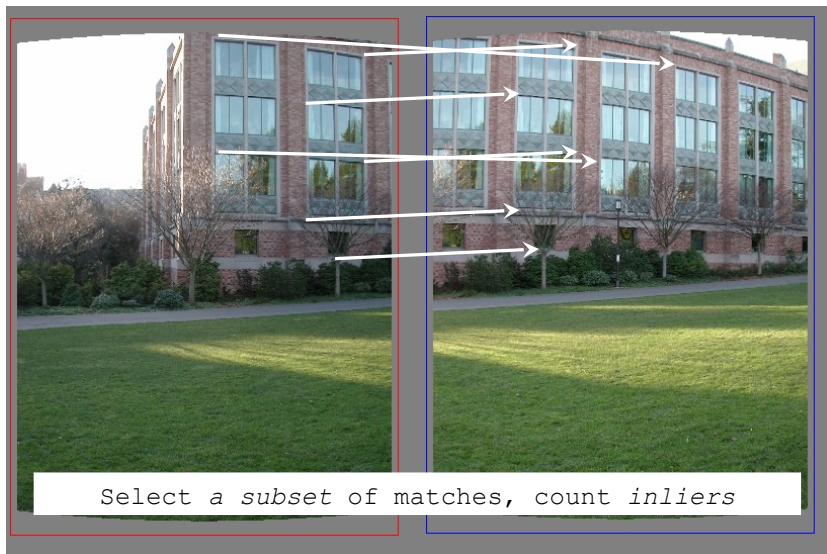


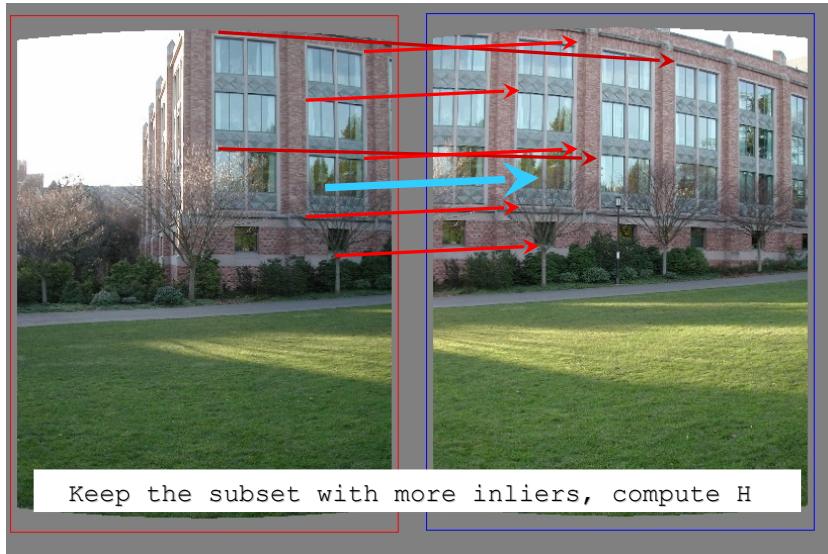
Quanto visto funziona bene se i match sono corretti, ma se non lo sono?



Applico un approccio RANSAC.

Ovvero provo un set, calcolo l'omografia e riproietto. Quanti sono gli inliers?





LA riproiezione che massimizza gli inliers è quella che tengo per buona



- Randomly select 4 matches
- Compute H from those matches
- Count how many inliers are obtained assuming
$$\|p'_2, H p_i\| < \varepsilon$$
- Repeat N times
- At the end recompute H using all inliers of the best match!

Riassunto



UNIVERSITÀ DI PARMA

Feature Matching

Question time!



Features Matching